

Previsão de readmissão hospitalar
Diabetes 130-US Hospitals (UCI 296)
IF1014 — Mineração de Dados Aplicada (CRISP-DM)

Equipe *João Pontes* *Marcos Antonio* *Theo Barza*

UFPE — 2026.1

27 de maio de 2026

Briefing do cliente

Cliente fictício: SI5 Health, rede hospitalar com unidade de cuidado pós-alta.

Problema. Identificar, na alta do paciente diabético, a probabilidade de **readmissão precoce** (<30 dias) para acionar acompanhamento ativo.

Saída. Classe $\in \{<30, >30, NO\}$.

Métrica de sucesso. F1-macro no conjunto de teste (dataset desbalanceado; <30 representa 11%).

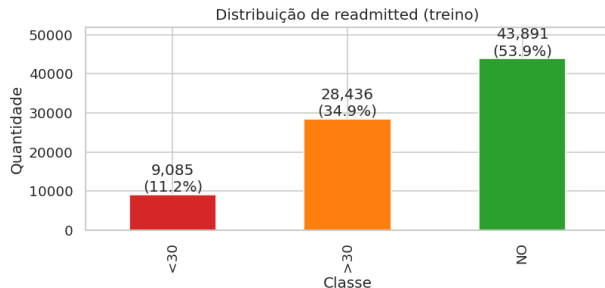
Restrições. Falso negativo da classe <30 é o erro mais caro (paciente readmite sem suporte). Limiar de operação calibrado pela capacidade do time de pós-alta.

Implicações éticas. Viés demográfico (race, gender, age), privacidade (dados pseudonimizados), automação de decisão clínica como apoio, não substituição.

Diabetes 130-US Hospitals (UCI 296, Strack et al., 2014).

- 101.766 internações → 81.412 treino / 20.354 teste (split estratificado, `random_state=42`).
- 47 features após remover IDs, `weight` (97% missing), `examide`, `citoglipton`.
- Categóricas dominam: 24 medicações + `raça`, `gênero`, `idade`, `diag_1/2/3` (ICD-9 agrupado), `payer_code`, `medical_specialty` (alta cardinalidade).
- Faltantes codificados como ? no original.

EDA — pontos relevantes



- Desbalanceamento: NO 54%, >30 35%, <30 11%.
- `number_inpatient`, `number_emergency` muito assimétricas → motivam variante **RobustScaler**.
- Missingness pesado em `race`, `payer_code`, `medical_specialty`, `diag_*`.
- ICD-9 com 1000+ códigos → agrupados em 9 categorias clínicas.

Pipeline baseline

Limpeza estrutural (RawCleaner): ? $\rightarrow$ NaN, drop de colunas inúteis, agrupamento ICD-9, IDs como string.

ColumnTransformer:

- Numéricas: `SimpleImputer(median)` $\rightarrow$ `StandardScaler`.
- Categóricas: `SimpleImputer(most_frequent)` $\rightarrow$ `OneHotEncoder(handle_unknown="ignore")`.

Crucial: todo `fit_transform` acontece **dentro de cada fold** da CV. Nada toca o conjunto de teste antes da Seção 9.

Variante: **SMOTE** e **Undersampling** (imblearn pipeline), **RobustScaler**, **TargetEncoder** em colunas de alta cardinalidade.

Os 10 algoritmos

Parte 1 (clássicos)	Parte 2 (avançados)
K-NN	MLP
LVQ (sklvq)	Comitê de MLPs (Bagging)
Árvore de Decisão	Stacking heterogêneo
SVM (LinearSVC)	XGBoost (GPU/CUDA)
Random Forest	LightGBM

Cada modelo definido em `src/models.py` via `ModelSpec` (factory + grid + estratégia de busca: Grid ou Randomized).

Busca de hiperparâmetros

Mesma CV em todos: `StratifiedKFold(n_splits=5, shuffle=True, random_state=42)`. Comparações pareadas exigem folds idênticos.

Métrica de otimização: `f1_macro`.

Estratégia: Grid em modelos com poucos HPs (DT, KNN, LVQ, Stacking, Comitê de MLPs); Randomized (`n_iter=5-10`) em modelos com espaço maior (RF, LightGBM, SVM, MLP, XGBoost).

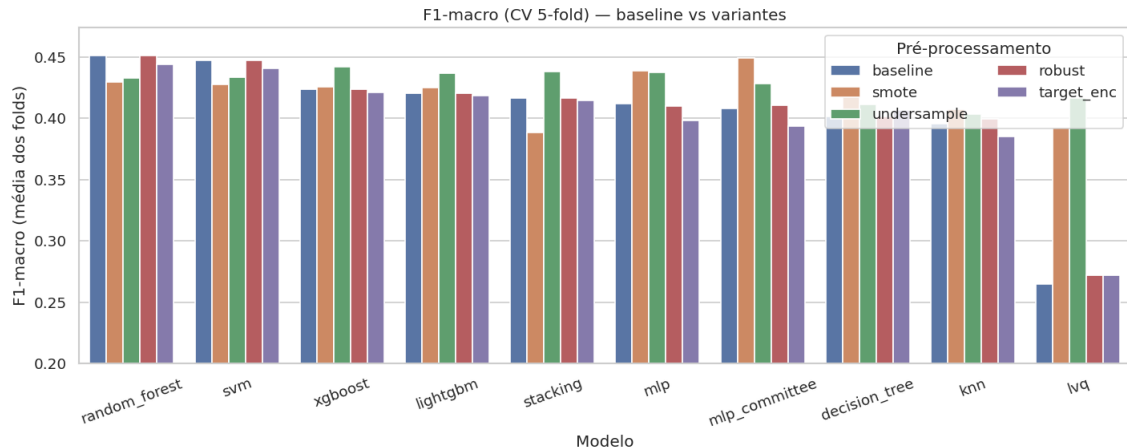
Curvas treino vs validação salvas por configuração em `reports/figures/<modelo>__<tag>__train_vs_val.png` (uma figura por modelo $\times$ pré-processamento).

Baseline — F1-macro por modelo (CV 5-fold no treino/validação)

Modelo	F1-macro CV	σ folds	Gap treino-val
Random Forest	0.4512	0.0028	0.39
SVM (LinearSVC)	0.4471	0.0024	0.01
XGBoost (GPU)	0.4240	0.0023	0.21
LightGBM	0.4204	0.0017	0.28
Stacking	0.4167	0.0021	0.06
MLP	0.4121	0.0115	0.02
Comitê de MLPs	0.4079	0.0055	0.05
Decision Tree	0.4012	0.0013	0.28
K-NN	0.3956	0.0020	0.19
LVQ (GLVQ)	0.2648	0.0024	0.00

Observação. RF e SVM lideram porque a busca selecionou `class_weight="balanced"` (já endereça desbalanceamento na fase de Modeling). O Comitê de MLPs (10 estimadores, bootstrap de 60% das amostras) fica no pelotão intermediário, junto do MLP isolado. LVQ é o mais fraco — protótipos sofrem em 200+ features OHE.

F1-macro: panorama visual



Cada barra = média dos 5 folds da melhor configuração de HP da combinação modelo × pré-processamento.
Eixo Y começa em 0.20 pra evidenciar variação.

Todos os valores deste gráfico são de validação cruzada (5 fold) no conjunto de treino, nenhum número de

Comparação baseline vs variantes — resultados significativos

Critério: $p < 0.05$ em paired t-test (5 folds pareados). Wilcoxon com 5 folds é *conservador* (p mín. = 0.0625, nunca atinge 0.05). **Todos os F1 desta tabela são de validação cruzada no treino, não do teste.**

Modelo	Variante	$F1_{base}$	$F1_{var}$	Δ	wilcoxon p	paired-t p
LVQ	Undersample	0.2648	0.4168	+0.152	0.062	<0.001*
LVQ	SMOTE	0.2648	0.3921	+0.127	0.062	<0.001*
Comitê de MLPs	SMOTE	0.4079	0.4490	+0.041	0.062	<0.001*
MLP	SMOTE	0.4121	0.4387	+0.027	0.062	0.005*
Stacking	Undersample	0.4167	0.4384	+0.022	0.062	<0.001*
Comitê de MLPs	Undersample	0.4079	0.4283	+0.020	0.062	0.001*
XGBoost	Undersample	0.4240	0.4419	+0.018	0.062	<0.001*
Decision Tree	SMOTE	0.4012	0.4190	+0.018	0.062	0.007*
LightGBM	Undersample	0.4204	0.4370	+0.017	0.062	0.001*
LVQ	TargetEnc	0.2648	0.2721	+0.007	0.062	0.001*
LVQ	RobustScaler	0.2648	0.2719	+0.007	0.062	0.003*
Decision Tree	TargetEnc	0.4012	0.4067	+0.006	0.062	0.042*
LightGBM	SMOTE	0.4204	0.4253	+0.005	0.062	0.016*
Stacking	SMOTE	0.4167	0.3885	-0.028	0.062	<0.001*
Random Forest	SMOTE	0.4512	0.4298	-0.021	0.062	<0.001*
SVM	SMOTE	0.4471	0.4277	-0.020	0.062	<0.001*
Random Forest	Undersample	0.4512	0.4328	-0.018	0.062	<0.001*
MLP	TargetEnc	0.4121	0.3980	-0.014	0.125	0.032*
SVM	Undersample	0.4471	0.4338	-0.013	0.062	<0.001*
K-NN	TargetEnc	0.3951	0.3854	-0.010	0.062	0.002*
Random Forest	TargetEnc	0.4512	0.4439	-0.007	0.062	0.005*
SVM	TargetEnc	0.4471	0.4410	-0.006	0.062	0.001*
XGBoost	TargetEnc	0.4240	0.4210	-0.003	0.062	0.045*

* = $p < 0.05$ em paired t-test. **Verde:** promovida. **Vermelho:** atrapalha.

Comparação baseline vs variantes — empate técnico ($p \geq 0.05$)

Diferenças abaixo do limiar de significância estatística — reportadas para completude.

Modelo	Variante	$F1_{base}$	$F1_{var}$	Δ	paired-t p
Decision Tree	Undersample	0.4012	0.4111	+0.010	0.151
XGBoost	SMOTE	0.4240	0.4259	+0.002	0.330
LightGBM	RobustScaler	0.4204	0.4208	+0.000	0.374
Decision Tree	RobustScaler	0.4012	0.4012	+0.000	0.933
SVM	RobustScaler	0.4471	0.4472	+0.000	0.130
Random Forest	RobustScaler	0.4512	0.4513	+0.000	0.916
XGBoost	RobustScaler	0.4240	0.4240	+0.000	—
LightGBM	TargetEnc	0.4204	0.4184	-0.002	0.050
MLP	RobustScaler	0.4121	0.4100	-0.002	0.827
Stacking	TargetEnc	0.4167	0.4145	-0.002	0.062
Stacking	RobustScaler	0.4167	0.4164	-0.000	0.474

Nota: LightGBM TargetEnc ($p=0.050$) está exatamente no limiar — tratar como empate técnico.

Contexto: F1 nesse dataset

Por que F1 ~ 0.45 é competitivo?

- Strack et al. (2014, paper original do dataset): **F1-macro ~ 0.43** .
- Trabalhos posteriores: 0.42–0.48 com feature engineering pesada.
- Acima de 0.55: **suspeito de leakage** (rótulo informa as features de algum jeito).

Por que é difícil?

- <30 vs >30 é uma fronteira *clínica arbitrária*. Paciente que readmite em 28 dias é praticamente indistinguível de um que readmite em 32 dias.
- Forte desbalanceamento: $<30 = 11\%$, $>30 = 35\%$, $NO = 54\%$.
- Sinais clínicos compartilhados: idade, número de visitas anteriores, diagnósticos crônicos elevam risco *nas duas* classes positivas.

Nossa contribuição não é F1 alto: é uma comparação *metodologicamente rigorosa* entre 10 algoritmos $\times$ 4 pré-processamentos, com Wilcoxon pareado para todas as decisões.

Achado metodológico: SMOTE não é gratuito

Padrão claro e simétrico entre SMOTE e Undersampling:

- **Reamostragem ajuda** modelos *sem balanceamento embutido*:
 - LVQ: +0.152 com undersample, +0.127 com SMOTE — *maiores ganhos absolutos do estudo*
 - **Comitê de MLPs: +0.041 com SMOTE** — sobe ao 2º melhor F1 geral (0.449)
 - MLP: +0.027 (SMOTE) / +0.025 (undersample); Decision Tree: +0.018 (SMOTE)
 - Boostings com undersample: XGBoost +0.018, LightGBM +0.017; Stacking +0.022
- **Reamostragem atrapalha** modelos com `class_weight="balanced"` (selecionado pela busca HP): Random Forest perde 0.021 (SMOTE) / 0.018 (under); SVM perde 0.020 / 0.013. Ambas reamostragens *deterioram* o classificador já calibrado.

Implicações.

- Em produção, escolher *uma* estratégia de desbalanceamento e medir, não combinar.
- Custo computacional: SMOTE adiciona ~30% de linhas sintéticas em cada fold. Para LVQ valeu (ganho 50%); para RF foi prejuízo de tempo e F1.

RobustScaler: efeito nulo em todos os modelos testados ($p \geq 0.05$). Esperado: árvores e modelos baseados em ranking são invariantes a escala; SVM e MLP já normalizados pelo baseline (StandardScaler).

TargetEncoder é heterogêneo: melhora DT e LVQ marginalmente (modelos sem representação sensível a valores), mas **degrada K-NN (-0.010, $p=0.002$) e MLP (-0.014, $p=0.032$)** — modelos sensíveis à escala e representação das features. TargetEncoding em colunas de alta cardinalidade (`diag_*`, `payer_code`) introduz leakage controlado dentro do fold mas reduz granularidade vs OHE para modelos de distância/ativação.

Modelo vencedor

Vencedor: **Random Forest** (com pré-processamento baseline ou RobustScaler — *empate técnico*).

F1-macro CV: **0.4513 $\pm$ 0.0028** (5 folds estratificados).

Hiperparâmetros (encontrados na busca):

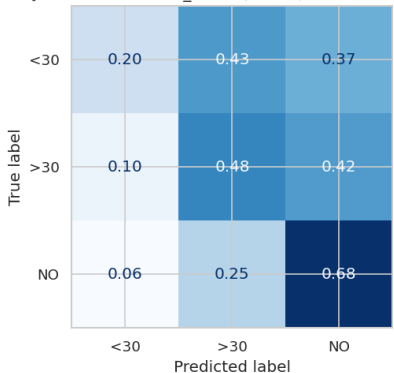
- `n_estimators=200`, `max_depth=20`, `max_features="log2"`
- `min_samples_leaf=1`, `class_weight="balanced"`

Por quê esse vence:

- **Top do ranking** dentre as **50** combinações avaliadas (10 modelos $\times$ 5 pré-proc.). 2º Comitê de MLPs+SMOTE (0.4490), 3º SVM (0.4471) — ambos empate técnico ($\Delta < 0.005$, sem significância).
- `class_weight="balanced"` ataca o desbalanceamento *na árvore*, sem custo de SMOTE.
- Curva treino vs val: overfit moderado ($F1_{tr}=0.84$, $F1_{val}=0.45$) mas *estável* entre folds ($\sigma=0.003$).
- Inferência rápida ($\sim$ ms/amostra), interpretabilidade via importância de features, pronto pra deployment.
- Variantes em RF: **SMOTE atrapalha** (-0.021 , $p < 0.001$), *Robust empata*, TargetEnc atrapalha (-0.007).

Avaliação final no teste

Avaliação final — random_forest (robust) — matriz normalizada



F1-macro teste: 0.4553

F1-macro CV: 0.4513 ± 0.0028

Δ teste — CV: +0.0041

Balanced accuracy: 0.4530

ROC-AUC macro: 0.6737

PR-AUC macro: 0.4739

recall(<30): 0.196

(métrica operacional crítica — 80% dos pacientes de alto risco não sinalizados)

Classe	Prec.	Rec.	F1
<30	0.241	0.196	0.216
>30	0.477	0.479	0.478
NO	0.661	0.684	0.672

Deployment e riscos

Uso. Score acionado na alta; pacientes <30 entram em fila de acompanhamento (ligação 48h, retorno 7d, revisão farmacoterapia).

Calibração. Threshold tuning sobre `predict_proba(<30)` para maximizar recall sujeito ao teto de fila operacional.

Riscos:

- Viés demográfico (race/age/gender). Mitigação: auditoria DIR por subgrupo, F1-macro estratificado mensal.
- Drift de população (nova unidade, mudança de mix). Mitigação: PSI semanal nas top-15 features.
- Privacidade: pseudonimização, sem retenção de raw fora de data/raw/.

Retreinamento. Gatilhos: $PSI > 0.25$ em ≥ 3 features, F1 mensal cai > 0.05 vs teste, ou trimestralmente.

1 comando de setup, 1 de execução.

```
uv sync
```

```
uv run jupyter nbconvert -to notebook -execute main.ipynb
```

Artefatos versionados: reports/cv_results/*.csv (cv_results_ completo por modelo × variante), reports/figures/*.png (curvas treino vs val, matriz, ROC, PR, barras).

Integridade do split: load_test levanta PermissionError sem o token I_AM_IN_FINAL_EVALUATION. Token aparece apenas na Seção 9 do notebook.

Uso de IA generativa. Declarado (Seção 14 das exigências): Claude Opus 4.7 usado para apoio em codificação, revisão e documentação. Decisões metodológicas (split, métrica, escolha de modelos e variantes, leitura dos resultados) conduzidas pela equipe.

Perguntas?

Repositório: <https://github.com/pontesjpp/project-readmission-diabetes.git>